TurboRUL

AI predictive maintenance for aircraft engines — project report
Status: Phase 1 of 6 complete (setup and exploratory analysis) · 06 October 2026

1. Summary

TurboRUL predicts the **Remaining Useful Life (RUL)** of turbofan engines, in flight cycles, from sensor histories, and turns those predictions into a ranked maintenance work list. It is built on NASA's public C-MAPSS run-to-failure dataset. This report covers the first phase: project setup, a tested data pipeline, and an exploratory analysis that fixes how sensors are selected and normalised before any model is trained. No model results exist yet; they arrive in Phases 2–3.

- **Data pipeline working and tested.** All four subsets (FD001–FD004) load with capped RUL labels; 14 unit tests pass.
- **7 of 21 sensors carry no signal in FD001** (s1, s5, s6, s10, s16, s18, s19), matching published practice.
- **Sensor selection must be per subset.** s10 is flat in FD001 but strongly informative in FD003.
- **Per-condition normalisation is essential for FD002/FD004**: it lifts sensor–RUL correlation from $|\rho| \approx 0.1$ to ≈ 0.7.
- **Two fault modes push five sensors in opposite directions** (FD003/FD004), which hides them from simple correlation and gives a ready signal for fault classification.
- **Data correction:** FD004 has 249 training and 248 test engines; the dataset readme swaps these counts.

2. Problem and objectives

Airlines maintain engines either on fixed schedules, which replace parts that still have life left, or after failures, which ground aircraft unexpectedly. Predicting cycles-to-failure lets planners schedule shop visits just in time, during planned downtime. Each prediction will carry an uncertainty range, a risk tier (Critical ≤ 30, Watch ≤ 60, otherwise Healthy), and the sensors that drove it.

Success metric	Target
RMSE on FD001 test set	≤ 15 cycles (stretch ≤ 13)
NASA asymmetric score	Reported; beats own baselines
Improvement over linear baseline	≥ 25% lower RMSE
Critical-engine recall (true RUL ≤ 30 flagged Critical)	≥ 90%
Coverage	RMSE reported on all four subsets
Demo latency	Prediction in < 2 s

3. Dataset

C-MAPSS simulates a fleet of large commercial turbofans. Each engine starts with unknown manufacturing wear, develops a fault, and is run cycle by cycle until a health margin crosses a failure threshold. Every cycle records 3 operating settings (altitude, Mach number, throttle) and 21 noisy sensors. Training files hold full run-to-failure histories; test files are cut off before failure, with the true RUL at the cut-off given separately. Counts below are measured from the files.

Subset	Train engines	Test engines	Operating conditions	Fault modes	Life (min / median / max)	Test histories < 30 cycles	Test RUL ≤ 30
FD001	100	100	1	HPC	128 / 199 / 362	0	25
FD002	260	259	6	HPC	128 / 199 / 378	6	61
FD003	100	100	1	HPC + fan	145 / 220 / 525	0	20
FD004	249	248	6	HPC + fan	128 / 234 / 543	11	53

Labels. Training rows get RUL = last cycle − current cycle, clipped at **125 cycles** (configurable). Early-life engines show no measurable degradation, so a piecewise-linear target is standard. Test rows are labelled backwards from the official RUL at their last cycle, which is the only point scored.

4. Work completed

Component	What it does
configs/default.yaml	Single source for RUL cap, window length, seeds, tier thresholds and paths (nothing hard-coded).
src/turborul/data.py	Parses the raw space-separated files (trailing spaces, no header), names the 26 columns, validates engine counts against RUL files, and labels train and test rows (FR-1).
tests/test_data.py	14 tests: countdown and cap logic, test-label back-fill, file parsing edge cases, and real-data checks (engine counts, RUL = 0 at failure, labels match official truth, contiguous cycles).
notebooks/01_eda.ipynb	Exploratory analysis below; uses training engines only for every sensor decision.
Environment	Python 3.11 virtual environment with pinned versions; git repository initialised.

5. Exploratory findings

5.1 Flat sensors

In FD001, six sensors are exactly constant and s6 takes only two values with no trend. The remaining 14 drift steadily towards failure, most sharply in the last ~100 cycles — exactly the region the RUL cap focuses the model on. s9 and s14 (core speed) are near-duplicates (r = 0.96) and rise in some engines while falling in others.

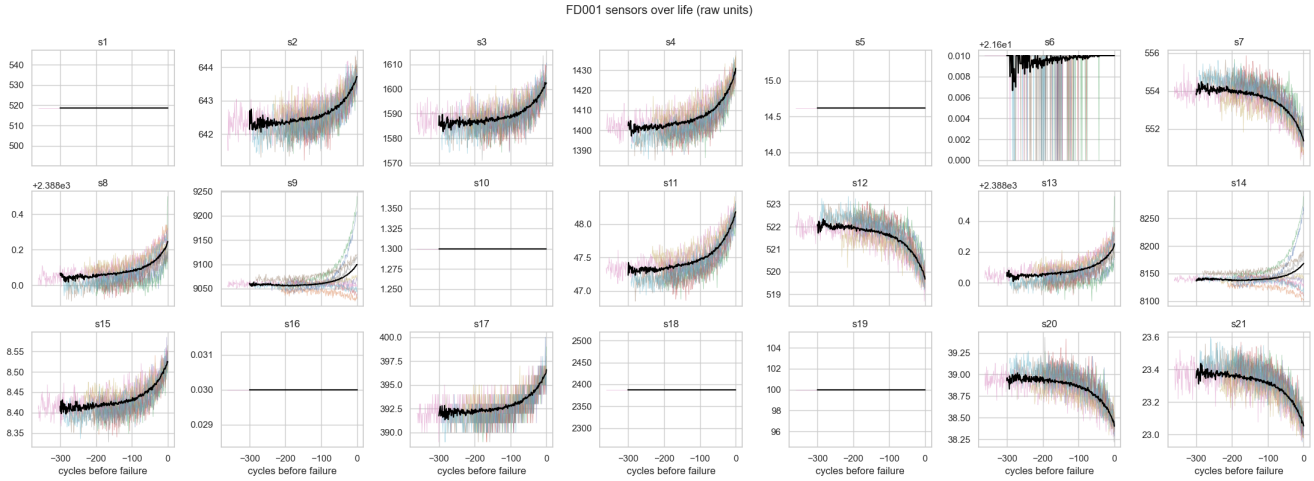


Figure 1. FD001 sensors aligned at failure: ten random engines (colour) and the fleet mean (black).

5.2 Operating conditions

FD002 and FD004 fly six distinct conditions, which k-means recovers with near-zero spread. Raw sensors jump between condition levels every cycle, swamping degradation. Standardising each sensor within its condition reveals the trend — this preprocessing step matters more than model choice on these subsets.

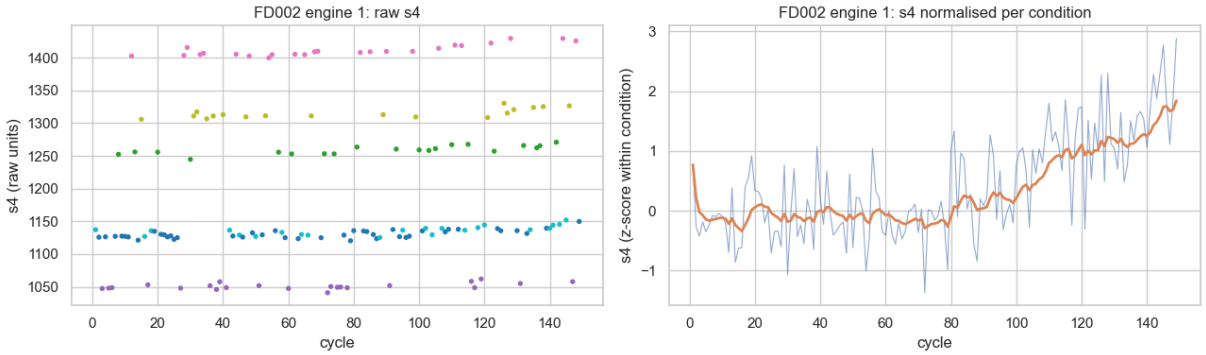


Figure 2. FD002 engine 1, sensor s4. Left: raw values coloured by operating condition. Right: after per-condition normalisation, with a smoothed trend.

5.3 Which sensors carry signal

Global correlation with RUL can be misleading, so each sensor is scored by **trend strength**: the mean, over engines, of |Spearman ρ| between the sensor and cycle within one engine. A second score, **direction agreement**, shows whether engines trend the same way.

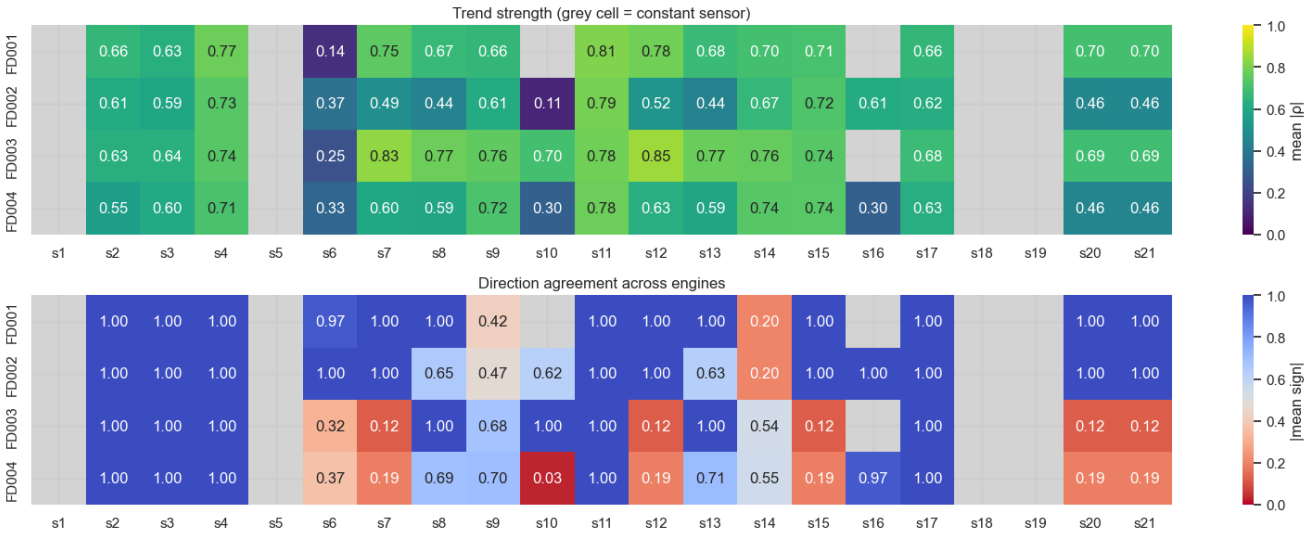


Figure 3. Trend strength (top) and direction agreement (bottom) per subset, after per-condition normalisation. Grey = constant sensor.

Selection rule (to be implemented in Phase 2, threshold in config): drop a sensor if it is constant within every condition or its trend strength is below 0.2.

Subset	Dropped sensors	Kept	Weak but kept (0.2–0.4)
FD001	s1, s5, s6, s10, s16, s18, s19	14	—
FD002	s1, s5, s10, s18, s19	16	s6
FD003	s1, s5, s16, s18, s19	16	s6
FD004	s1, s5, s18, s19	17	s6, s10, s16

5.4 Fault modes split sensor directions

In FD003 and FD004, s7, s12, s15, s20 and s21 rise towards failure in some engines and fall in others (56 vs 44 engines in FD003). The two groups correspond to the two fault modes. Their global correlation with RUL is near zero, yet per engine they are among the strongest signals. Models therefore need per-engine trend features (slopes) as well as levels, and these sensors make fault classification (FR-13) straightforward.

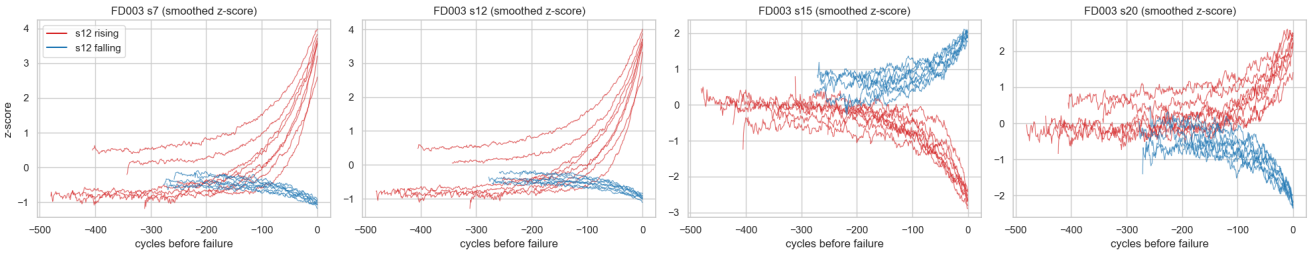


Figure 4. FD003 engines grouped by whether s12 rises (red) or falls (blue) over life.

6. Decisions carried into modelling

- Select sensors per subset with the trend-strength rule; never with a hard-coded list.
- Cluster operating conditions with k-means and normalise within condition for FD002/FD004, fitting both on training engines only.
- Include slope features alongside mean, std, min, max and last value for classical models.
- Pad short test histories: 6 FD002 and 11 FD004 test engines have fewer than 30 cycles.
- Expect coarse critical-recall steps on FD001: only 25 test engines have true RUL ≤ 30, so each counts for 4 points.
- Expect correlated sensors (e.g. s9/s14) to share SHAP credit and to destabilise linear-model coefficients.

7. Next steps

Phase	Weeks	Scope	Exit criterion
2 · Baselines	3–4	Preprocessing, engine-level split, 30-cycle windows, features, linear + XGBoost, metrics, training CLI	FD001 test RMSE for both models in results.csv
3 · Deep models	5–6	LSTM and 1D-CNN (PyTorch), Optuna tuning on validation engines, 5-seed runs	Deep model beats XGBoost or the gap is explained
4 · Robustness	7–8	FD002–FD004, uncertainty ranges, risk tiers, SHAP explanations	RMSE on all four subsets
5 · Product	9–10	FastAPI prediction endpoint, three-screen Streamlit dashboard	All screens working locally
6 · Showcase	11–12	Deployment, README, course report, demo video	Public demo live

8. Limitations

- The data is simulated, not from a real fleet, and failure is a threshold crossing rather than a removal decision.
- There is no cost, schedule or parts data, so maintenance value can only be argued, not measured.
- Test engines with true RUL above the cap (11 of 100 in FD001) cannot be predicted exactly by a capped model.
- Findings so far are descriptive; no predictive claims are made until models are scored once on the held-out test set.